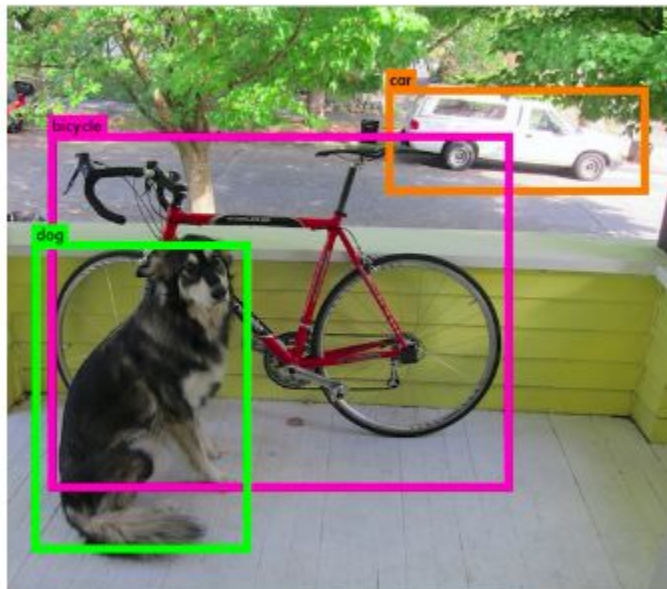


DETR(DEtection  
TRansformer)

# The last evil thing in object detection-NMS

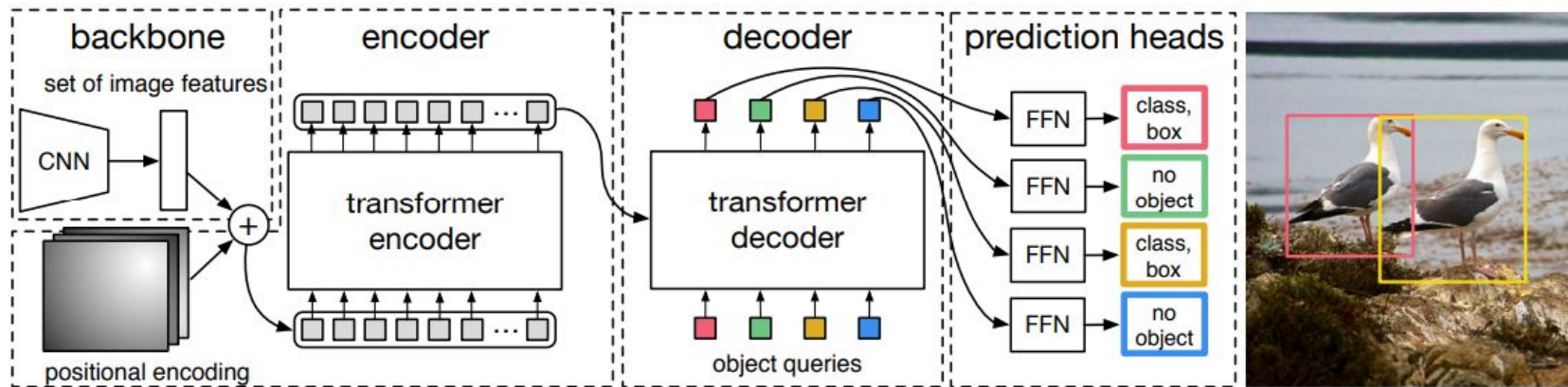


Multiple Bounding Boxes



Final Bounding Boxes

# DEtection TRnasformer



- This is released before vision transformer
- It use CNN to extract features and use transformers to produce predictions
- Encoder/Decoder paradigm

# Predict set

- Typically, we can use multilabel to represent “set” in deep learning.
  - replace the softmax with a set of element-wised sigmods
- The central issue is to avoid duplicated prediction
  - Different boxes can be very similar
  - Post processing, such as NMS is required
- Can we let model perform NMS?

# What's NMS?

1. Sort all boxes by area
2. Iterate from the biggest box
3. Check all boxes smaller than it
  - a. If the overlapped area is larger than a pre-defined threshold, remove the smaller one.

Why do we need this?

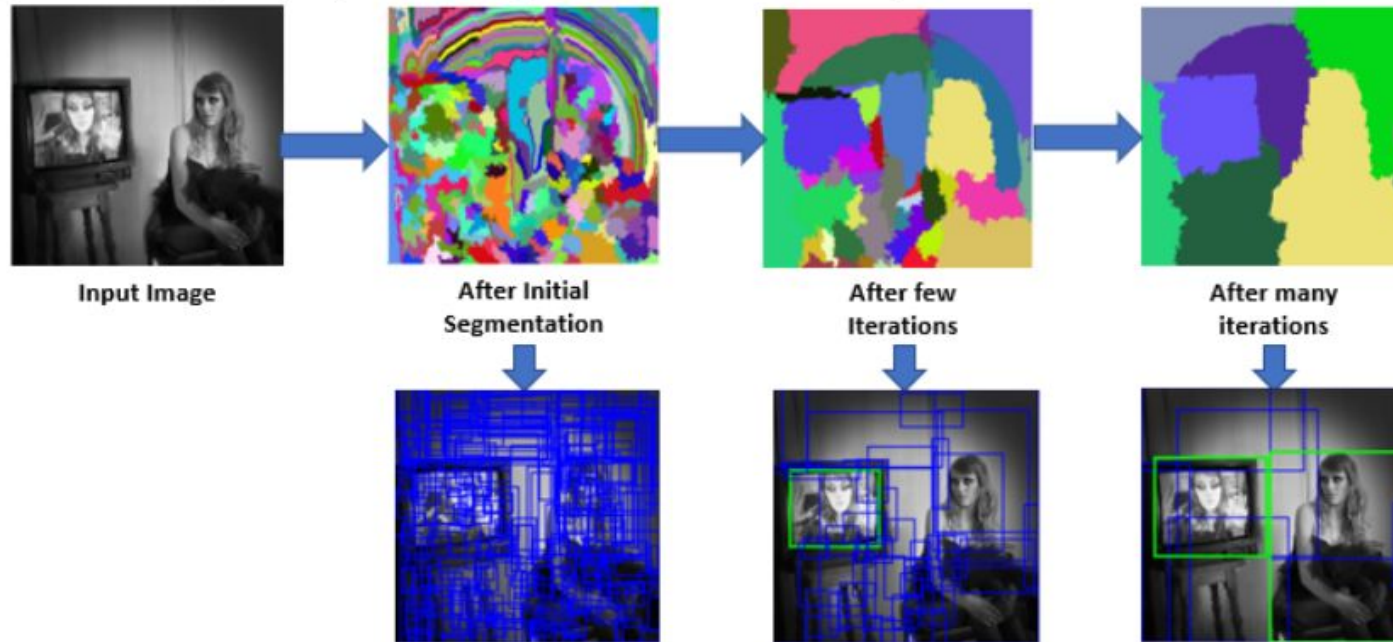
# How could we know the number of objects in a picture



We can have only one object in a picture.

Or we can have 100 objects in a picture.

# The selective search in the original RCNN

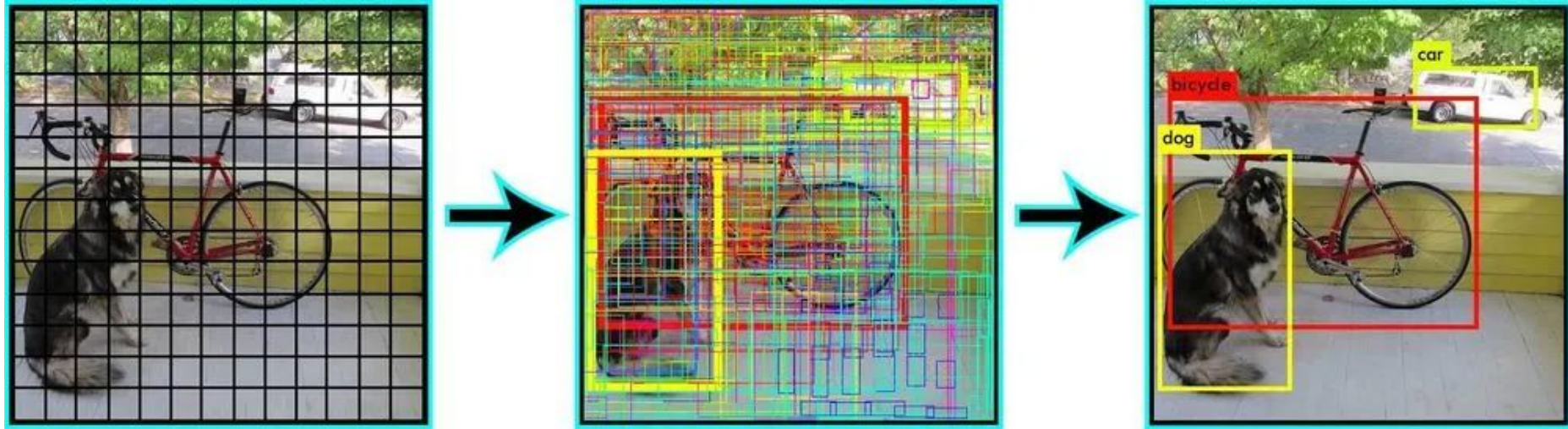


*Image showing segmentation and their candidate proposals (Source: selective Search Paper)*

We will generate thousands of **region proposals** and decide if they are objects or not.



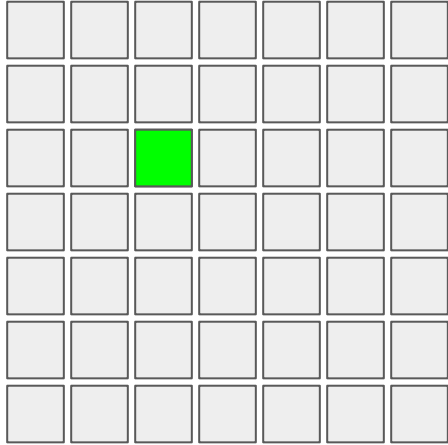
The model will generate multiple predictions for each objects.



There is no mechanism to prevent different predictions for the same object.



# YOLO use the dense prediction to replace it.



We have 7x7 grids in the final feature map.

For each grid, we can generate multiple predictions, whose center is in this grid.

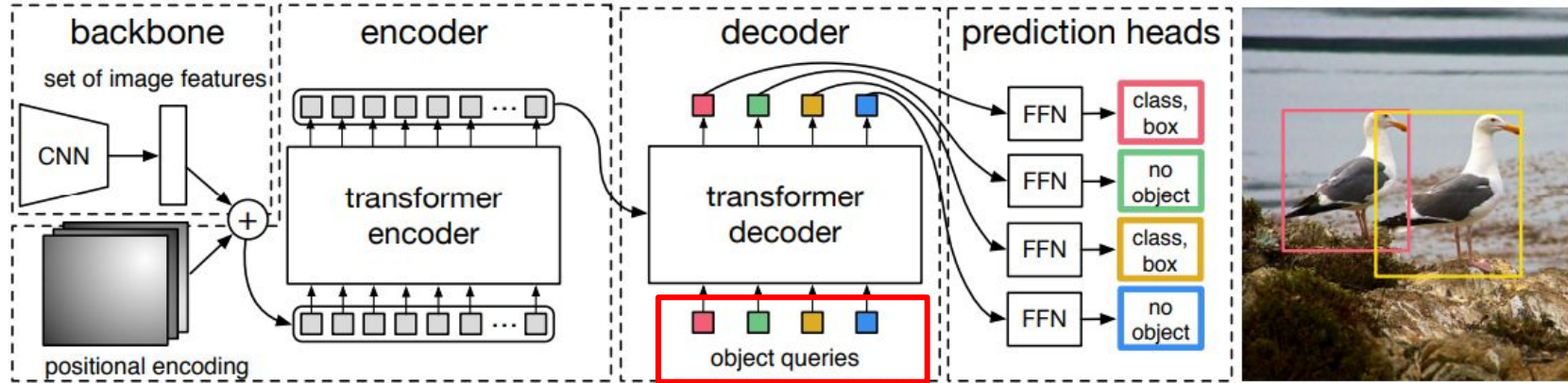
We may have more than two predictions, which are for the same objects.

Therefore, we need to use NMS to remove the duplicated predictions.

# Two issues which must be fixed

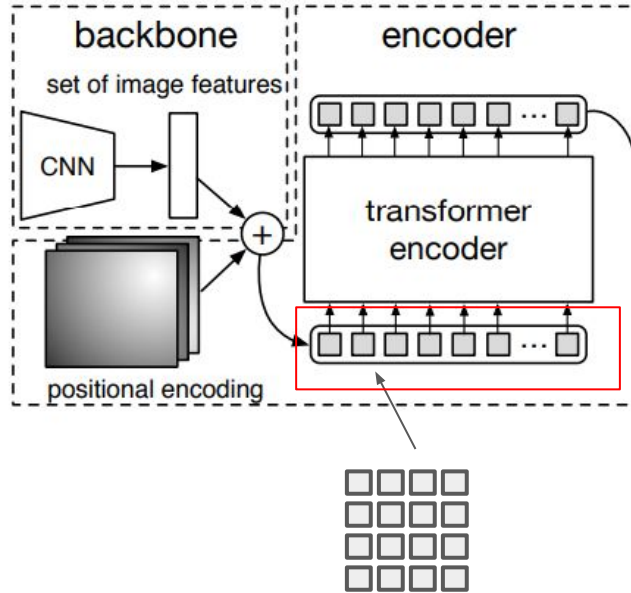
- dense predictions
  - Reduce the number of predictions.
  - Remove the grid and anchor.
- Prevent **duplicate predictions** for the same object

# Solve the number of predictions



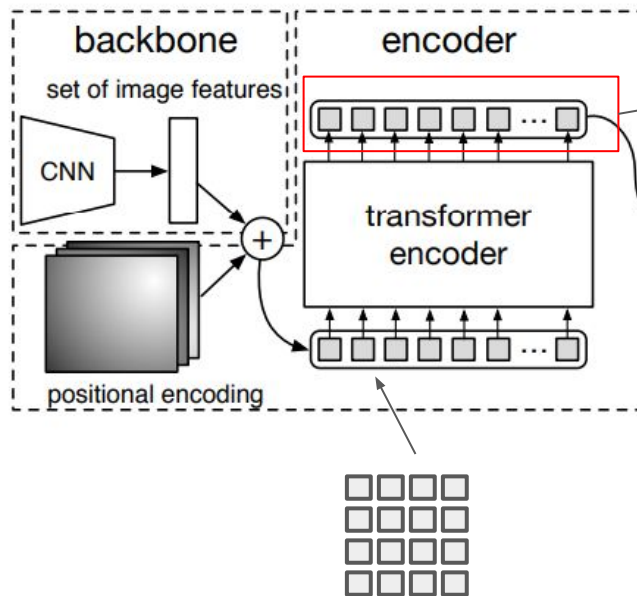
Each object query will generate one predictions at any location.

# How do we **detect** an object?



Every input input of the encoder represent a small portion of image in the input image.

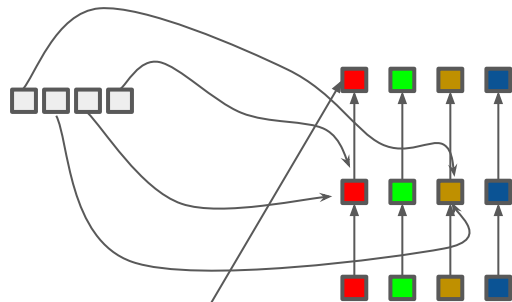
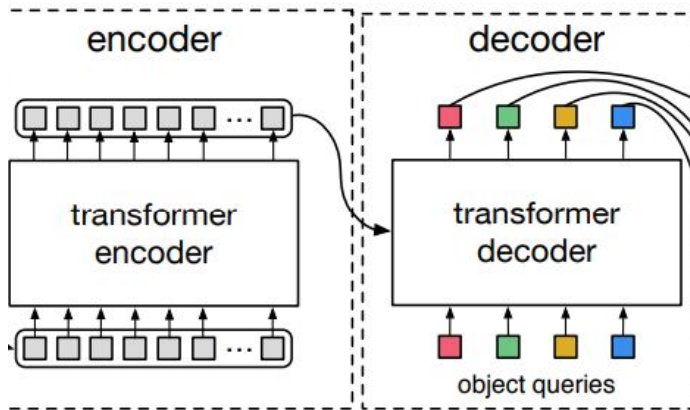
# How do we **detect** an object?



Every output of the encoder collect global information of the relationship of this patch to all other patches.

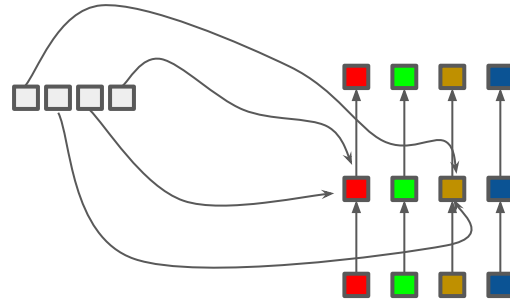
It is not “we have object here”. Every output is still a context of the entire picture for a specific patch.

# Dispatch every patch to an object



Every query is a **slot** of an object. The decoder will assign every output of encoder to a specific query by using cross attention.

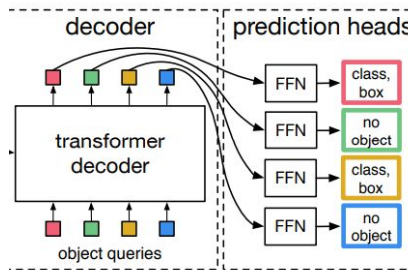
The output of decoder contains **information** from the encoder for an object.



We need to force the **information** of different **objects** to different **queries**.



# Prediction without duplication



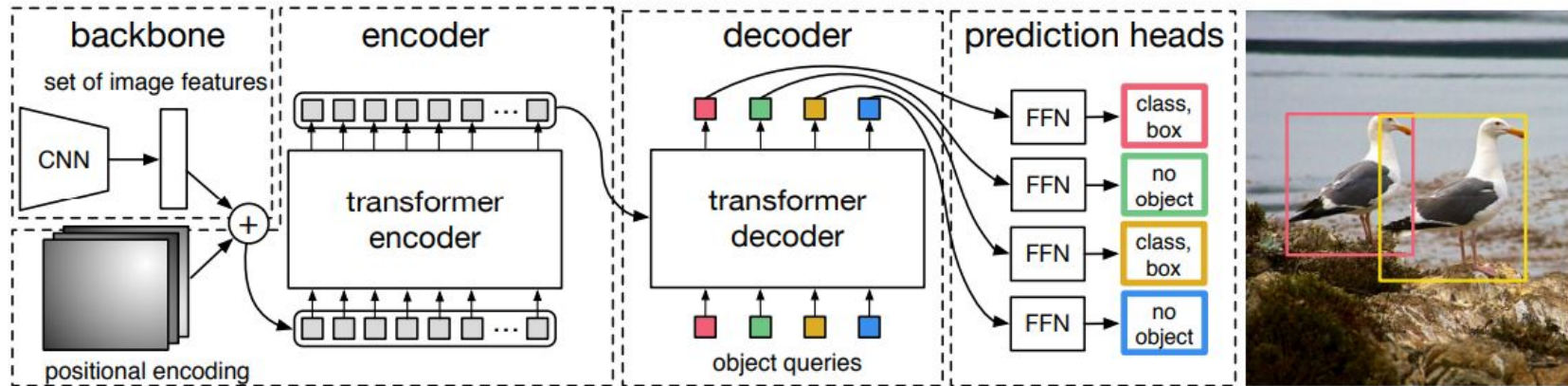
- Hungarian Algorithm
  - When a “set-to-set” matching is required (for example, establishing a one-to-one correspondence between an input set and an output set), the problem becomes minimizing the total matching error.
- The model can output a **set** of predictions and we have ground truth **set** for the input.
  - The loss is the error in the hungarian algorithm.
- We need to construct the cost matrix

$$C_{ij} = \lambda_{cls} \cdot L_{cls} (y_i, \hat{y}_j) + \lambda_{box} \cdot L_{box} (b_i, \hat{b}_j)$$

- Find the best match by using hungarian algorithm and calculate  $L = \sum_{i,j \in \pi} C_{ij}$

$$\pi = \arg \min_{\pi'} \sum_{i,j \in \pi'} C_{ij}$$

# parallel decoding



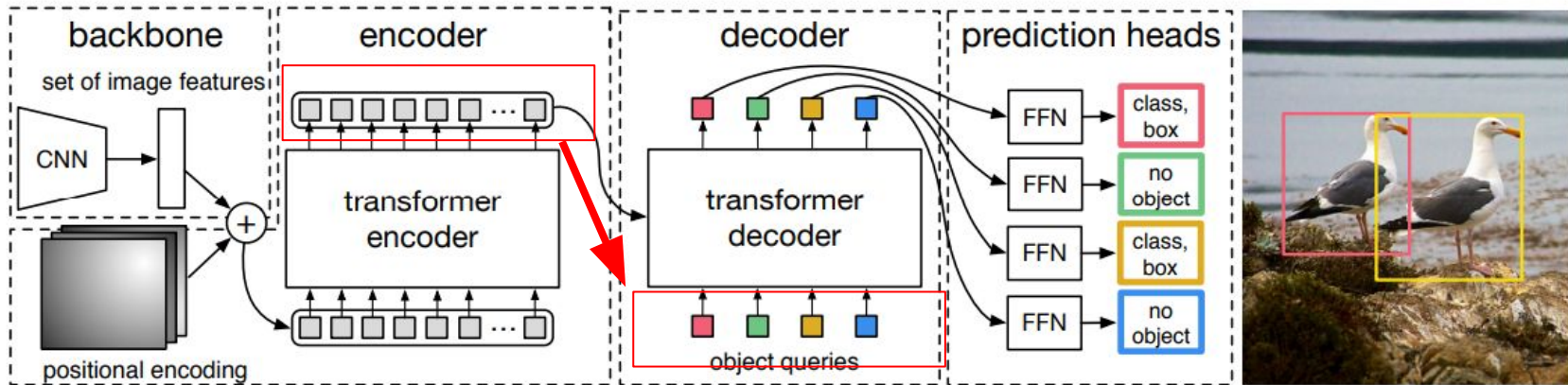
- Most prior work use autoregressive decoder(RNN) so that it can emulate the traditional algorithm to perform the job step by step.
-

# Issues in DETR

- It's innovative to drop NMS completely. However, it is not SOTA.
  - Finally, we remove all non-NN steps
  - No anchor, No NMS
- It is hard to be trained.
- It does not perform well for small objects.

# The training of DETR is difficult

How can we decide?

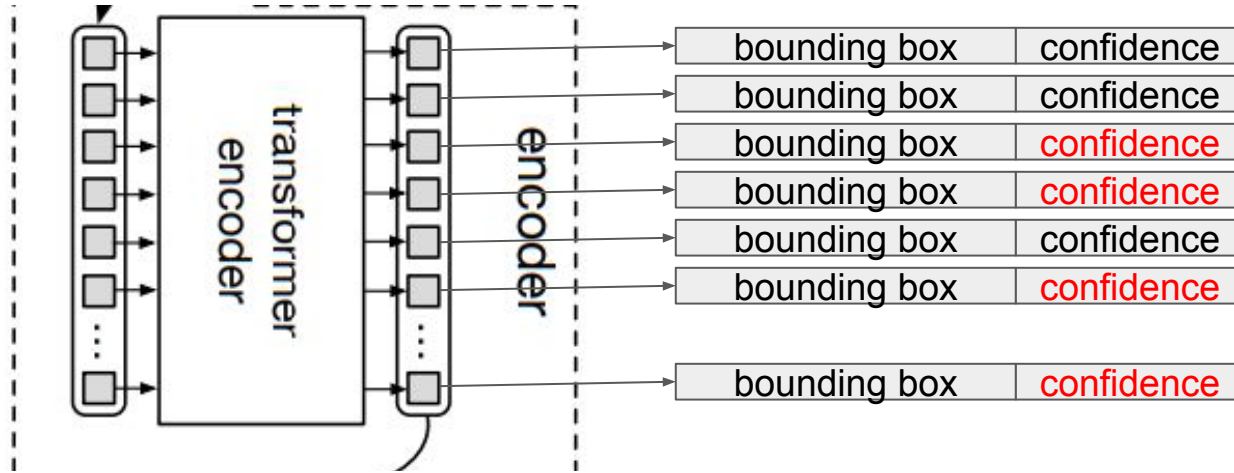


We need to learn the following in the same time

- what should I look for?
- where should I locate it?

It's better if we can tell the query the **clear target**(what should it look for)

# Prediction head in the stage 1



We will choose the top-k to the the queries for the stage 2.

## Two stage has better performance

| Method                              | Epochs | AP   | AP <sub>50</sub> | AP <sub>75</sub> | AP <sub>S</sub> | AP <sub>M</sub> | AP <sub>L</sub> | params | FLOPs | Training<br>GPU hours | Inference<br>FPS |
|-------------------------------------|--------|------|------------------|------------------|-----------------|-----------------|-----------------|--------|-------|-----------------------|------------------|
| Faster R-CNN + FPN                  | 109    | 42.0 | 62.1             | 45.5             | 26.6            | 45.4            | 53.4            | 42M    | 180G  | 380                   | 26               |
| DETR                                | 500    | 42.0 | 62.4             | 44.2             | 20.5            | 45.8            | 61.1            | 41M    | 86G   | 2000                  | 28               |
| DETR-DC5                            | 500    | 43.3 | 63.1             | 45.9             | 22.5            | 47.3            | 61.1            | 41M    | 187G  | 7000                  | 12               |
| DETR-DC5                            | 50     | 35.3 | 55.7             | 36.8             | 15.2            | 37.5            | 53.6            | 41M    | 187G  | 700                   | 12               |
| DETR-DC5 <sup>+</sup>               | 50     | 36.2 | 57.0             | 37.4             | 16.3            | 39.2            | 53.9            | 41M    | 187G  | 700                   | 12               |
| Deformable DETR                     | 50     | 43.8 | 62.6             | 47.7             | 26.4            | 47.1            | 58.0            | 40M    | 173G  | 325                   | 19               |
| + iterative bounding box refinement | 50     | 45.4 | 64.7             | 49.0             | 26.8            | 48.3            | 61.7            | 40M    | 173G  | 325                   | 19               |
| ++ two-stage Deformable DETR        | 50     | 46.2 | 65.2             | 50.0             | 28.8            | 49.2            | 61.7            | 40M    | 173G  | 340                   | 19               |

## Another issue: resolution

- The transformer does not work well for high resolution image because of the complexity.
-